

CIS 613: Path Testing

1

What is Path Testing?

- Code-Based (or Structural) Testing
- Complements Specification-Based (or Functional) Testing
- It is:
 - Based on the implementation,
 - Basis for coverage metrics, and
 - Usually done at unit level.

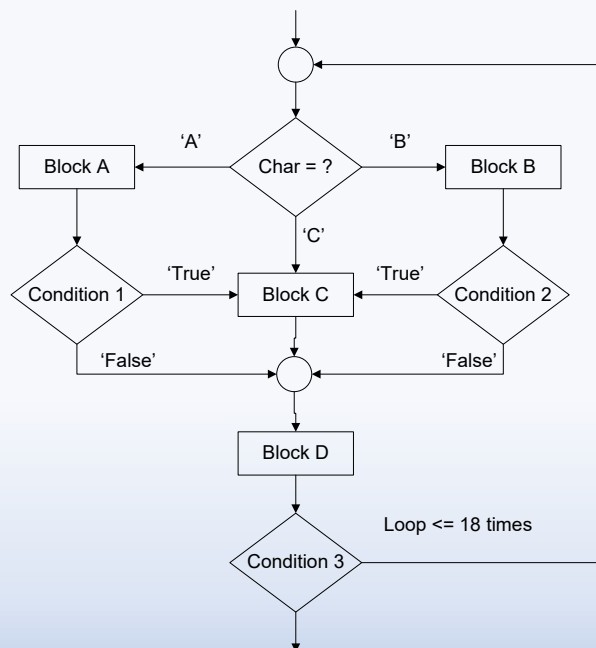
2

Path Testing

- Paths derived from some graph construct, typically representing executable code.
- When a test case executes, it traverses a path.
- Problem? Huge number of paths implies some simplification is needed.

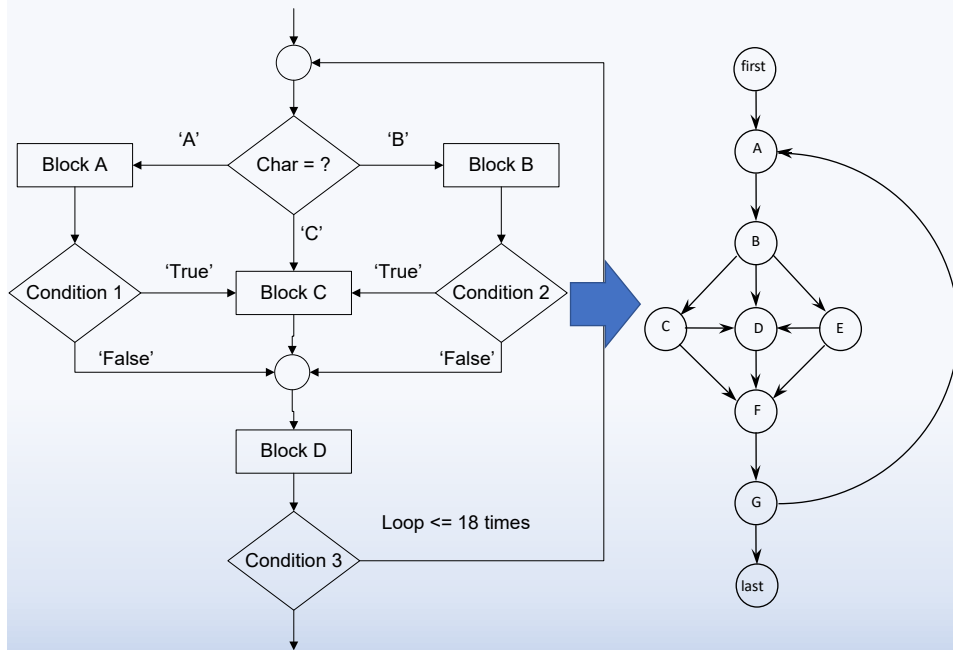
3

Common Objection to Path Testing



4

Common Objection to Path Testing

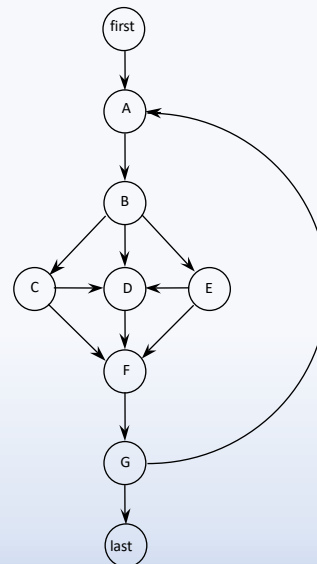


5

Common Objection to Path Testing

- If the loop executes up to 18 times, there are 4.77 Trillion paths. Impossible, or at least, infeasible, to test them all.*
- How do we get to 4.77 Trillion paths?
- $5^0 + 5^1 + 5^2 + \dots + 5^{18} = 4,768,371,582,030$

Can we get good coverage without 4.77 Trillion paths?



*Stephen R. Schach, *Software Engineering*, (2nd edition)

6

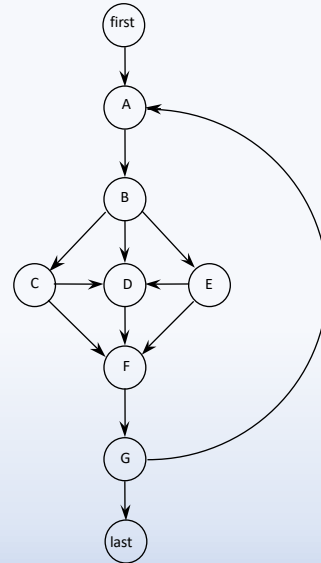
Common Objection to Path Testing

Test Cases:

1. First-A-B-C-F-G-Last
2. First-A-B-C-D-F-G-Last
3. First-A-B-D-F-G-A-B-D-F-G-Last
4. First-A-B-E-F-G-Last
5. First-A-B-E-D-F-G-Last

These test cases cover

- Every node
- Every edge
- Normal repeat of the loop
- Exiting the loop



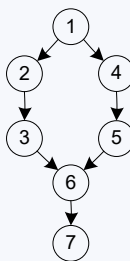
7

Reminder: Program Graphs

If-Then-Else

```

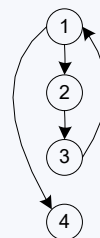
1 If <condition>
2   Then
3     <then statements>
4   Else
5     <else statements>
6 End If
7 <next statement>
  
```



Pre-test Loop

```

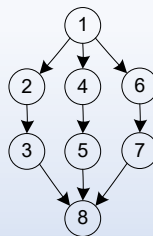
1 While <condition>
2   <repeated body>
3 End While
4 <next statement>
  
```



Case/Switch

```

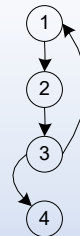
1 Case n Of 3
2   n=1:
3     <case 1 statements>
4   n=2:
5     <case 2 statements>
6   n=3:
7     <case 3 statements>
8 End Case
  
```



Post-test Loop

```

1 Do
2   <repeated body>
3 Until <condition>
4 <next statement>
  
```



8

Program Graph: In Class Exercise

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }

```

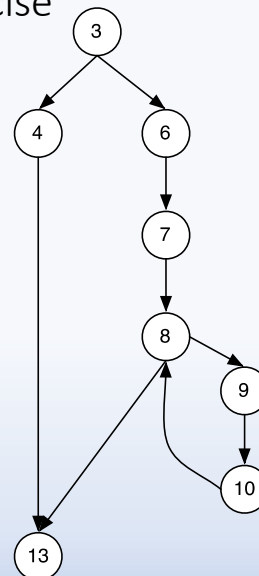
9

Program Graph: In Class Exercise

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }

```



10

Program Graph: In Class Exercise

```

1. def factorial(n):
2.     fact = None
3.     if n < 0:
4.         fact = 0
5.     else:
6.         i = 1
7.         fact = 1
8.         while i <= n:
9.             fact = fact * i
10.            i = i + 1
11.     return fact

```

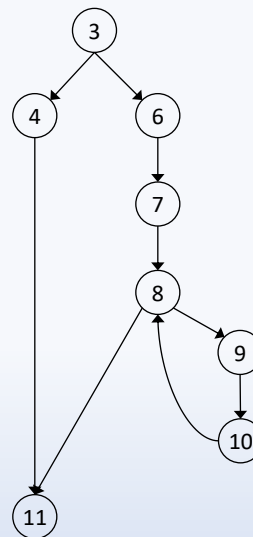
11

Program Graph: In Class Exercise

```

1. def factorial(n):
2.     fact = None
3.     if n < 0:
4.         fact = 0
5.     else:
6.         i = 1
7.         fact = 1
8.         while i <= n:
9.             fact = fact * i
10.            i = i + 1
11.     return fact

```



12

Basic Graph Coverage Metrics

- G_{node} : Every node in the graph is executed
- G_{edge} : Every edge in the graph is traversed during execution
- G_{chain} : Every chain in the graph is executed

13

More Ideas for Coverage of Program Graphs

E. F. Miller's Metrics:

- C_0 : Every statement
- C_1 : Every DD-Path
- C_{1p} : Every predicate outcome
- C_2 : C1 coverage + loop coverage
- C_d : C1 coverage + every pair of dependent DD-Paths
- C_{MCC} : Multiple condition coverage
- C_{ik} : Every program path that, up to k repetitions of a loop
- C_{stat} : "Statistically significant" fraction of paths
- C_{∞} : All possible execution paths

14

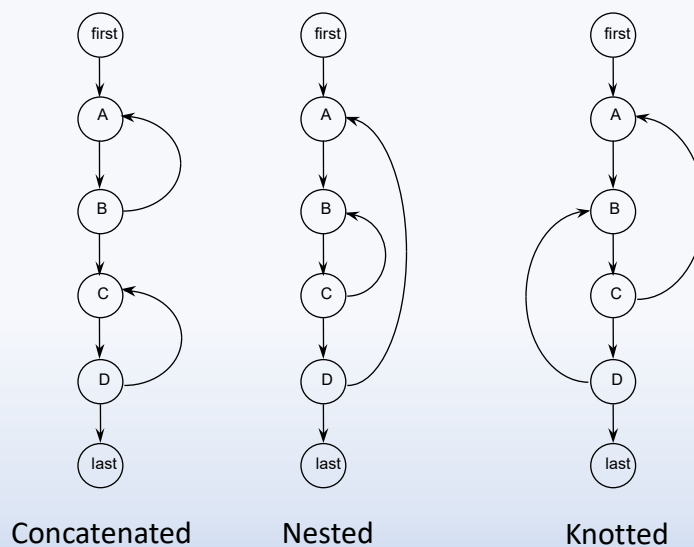
How many times do we test a loop?

- Normally twice (e.g., C_{jk} where $k = 2$). Why?
- **Huang's Theorem:** (Paraphrased) Everything interesting will happen in two loop traversals: the normal loop traversal and the exit from the loop.

There are different kinds of loops, of course.

15

Types of Loops



16

Testing Loops

- Huang's theorem *suggests* 2 tests per loop is sufficient.
- For **nested loops**:
 - Test innermost loop first
 - Then “condense” the loop into a single node (i.e., condensation graph)
 - Work from innermost to outermost loop
- For **concatenated loops**: use Huang's Theorem
- For **knotted loops**: Rewrite!

17

Basis Path Testing

18

Basis Path Testing

- Proposed by Thomas McCabe in 1982
- Math background
 - a Vector Space has a set of independent vectors called basis vectors
 - every element in a vector space can be expressed as a linear combination of the basis vectors
- Example: Euclidean 3-space has three basis vectors
 - $(1, 0, 0)$ in the x direction
 - $(0, 1, 0)$ in the y direction
 - $(0, 0, 1)$ in the z direction
- The Hope: If a program graph can be thought of as a vector space, there should be a set of basis vectors. Testing them tests many other paths.

19

Basis Path Testing

- A white-box testing technique first proposed by Tom McCabe.
- Derive a logical complexity measure of a procedural design.
- Use cyclomatic complexity to identify execution paths and design test cases to exercise these paths.
- Steps in basis path testing:
 1. Use the design or code as a foundation to draw a corresponding flow graph.
 2. Determine the cyclomatic complexity of the resulting flow graph.
 3. Determine a basis set of linearly independent paths.
 4. Prepare test cases that will force execution of each path in the basis set.

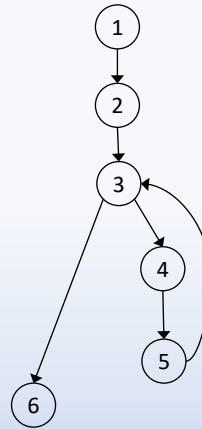
20

Basis Path Testing

- **Step 1:** Use the design or code as a foundation to draw a corresponding flow graph.

```
def sum_numbers(n):
    1 → i = 1
    2 → total = 0

    3 → while i <= n:
    4 →     total += 1
    5 →     i += 1
    6 → return total
```



21

Basis Path Testing

- **Step 2:** Determine the cyclomatic complexity of the resulting flow graph.
 - Cyclomatic complexity is a software metric that provides a quantitative measure of the logical complexity of a program.
 - When used in the context of the basis path testing, the value computed for cyclomatic complexity defines the number of *independent paths* through a program.
 - It provides an upper bound on the number of test cases that must be conducted to ensure all statements have been executed at least once.
 - Three ways of computing cyclomatic complexity:
 - Enclosed Regions Method,
 - Edge-Node Method,
 - Predicate-Node Method.

22

Basis Path Testing

- Enclosed Regions Method:

The cyclomatic complexity, $V(G)$, for a flow graph G is defined as

$$V(G) = ER + 1$$

where ER is the number of enclosed regions in the flow graph. Areas bounded by edges and nodes are called *enclosed regions*.

- Edge-Node Method:

The cyclomatic complexity, $V(G)$, for a flow graph G is defined as

$$V(G) = E - N + 2$$

where E is the number of edges and N is the number of nodes in the flow diagram.

23

Basis Path Testing

- Predicate-Node Method:

The cyclomatic complexity, $V(G)$, for a flow graph G is defined as

$$V(G) = P + 1$$

where P is the number of predicate nodes in the flow graph. A predicate node is a node with two or more edges emanating from it.

24

Basis Path Testing

- Cyclomatic complexity of **sum_numbers** function:

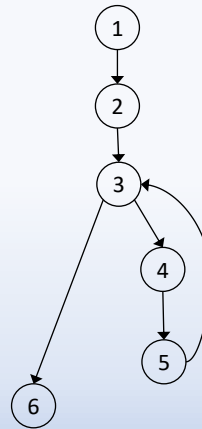
- $V(G) = ER + 1 = 1 + 1 = 2$
- $V(G) = E - N + 2 = 6 - 6 + 2 = 2$
- $V(G) = P + 1 = 1 + 1 = 2$

```
def sum_numbers(n):
```

```
1 → i = 1
2 → total = 0

3 → while i <= n:
4 →     total += 1
5 →     i += 1

6 → return total
```



25

Basis Path Testing

- **Step 3:** Determine a basis set of linearly independent paths.
 - An *independent path* must move along at least one edge that has *not* been traversed before the path is defined.
 - The set of independent paths is called its *basis set*
 - Independent path for **sum_numbers** function:
- **Step 4:** Prepare test cases that will force execution of each path in the basis set.
 - Path 1: Input: $n \leq 0$
Expected result: sum = 0
 - Path 2: Input $n \geq 1$
Expected result: sum of 1 to n

26

Basis Path Testing: In Class Exercise

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }

```

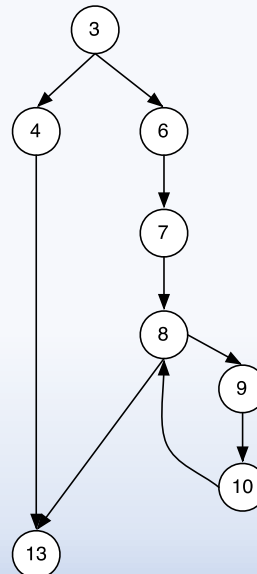
27

Basis Path Testing: In Class Exercise

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }

```



28

DD-Paths

- A decision-to-decision path (DD-Path) is a chain in a program graph, such that:
 1. it consists of a single node with indeg = 0,
 2. it consists of a single node with outdeg = 0,
 3. it consists of a single node with indeg ≥ 2 or outdeg ≥ 2 ,
 4. it consists of a single node with indeg = 1 and outdeg = 1,
 5. it is a maximal chain of length ≥ 1 .

What does this *really* mean?

DD-Paths support making a condensation graph.

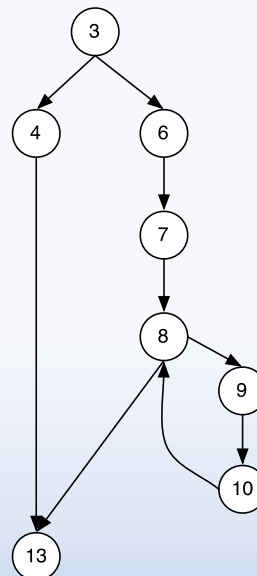
29

Basis Path Testing: In Class Exercise

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.         }
12.     }
13.     return fact;
14. }

```



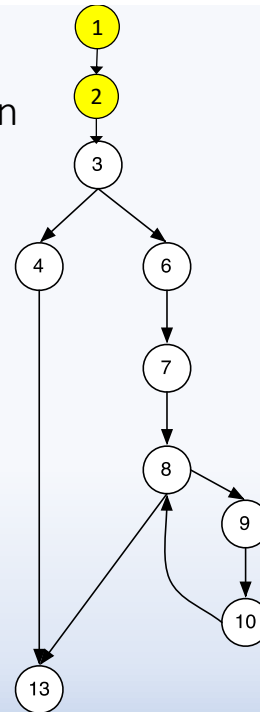
30

Comparison

```

1.  int factorial(int n) {
2.      int i, fact;
3.      if(n < 0) {
4.          fact = 0;
5.      } else {
6.          i = 1;
7.          fact = 1;
8.          while(i <= n) {
9.              fact = fact * i;
10.             i = i + 1;
11.          }
12.      }
13.      return fact;
14. }

```



31

Basis Path Testing & DD-Paths



If a program flow diagram is condensed into a condensation graph with DD-Paths, does this impact / change the number of linearly independent paths (i.e., cyclomatic complexity)?

- Yes!
- No!

32

Program Commission (INPUT,OUTPUT)

```
Dim locks, stocks, barrels As Integer
Dim lockPrice, stockPrice, barrelPrice As Real
Dim totalLocks, totalStocks, totalBarrels As Integer
Dim lockSales, stockSales, barrelSales As Real
Dim sales, commission : REAL
```

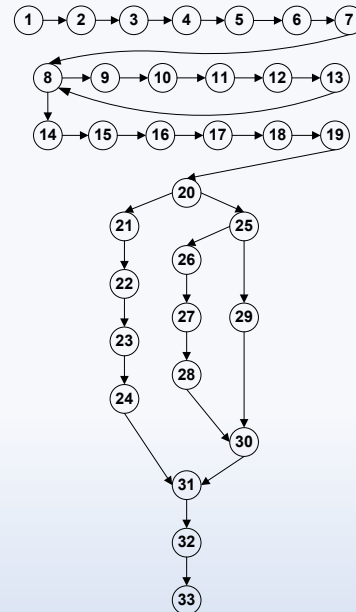
```
lockPrice = 45.0
stockPrice = 30.0
barrelPrice = 25.0
totalLocks = 0
totalStocks = 0
totalBarrels = 0
```

```
Input(locks)
While NOT(locks = -1) 'Input device uses -1 to indicate end of data
    Input(stocks, barrels)
    totalLocks = totalLocks + locks
    totalStocks = totalStocks + stocks
    totalBarrels = totalBarrels + barrels
    Input(locks)
EndWhile
```

```
Output("Locks sold: ", totalLocks)
Output("Stocks sold: ", totalStocks)
Output("Barrels sold: ", totalBarrels)
```

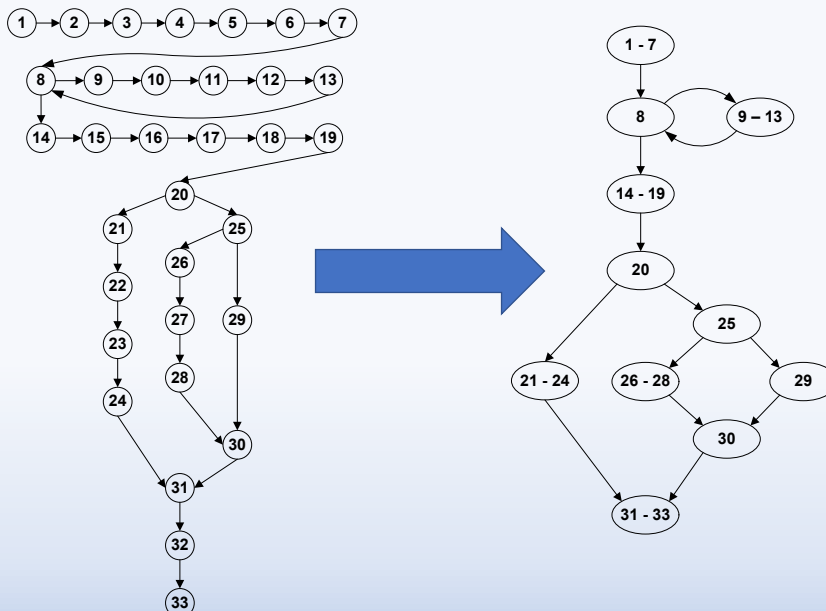
```
lockSales = lockPrice * totalLocks
stockSales = stockPrice * totalStocks
barrelSales = barrelPrice * totalBarrels
sales = lockSales + stockSales + barrelSales
Output("Total sales: ", sales)
```

```
If (sales > 1800.0)
    Then
        commission = 0.10 * 1000.0
        commission = commission + 0.15 * 800.0
        commission = commission + 0.20 * (sales - 1800.0)
    Else If (sales > 1000.0)
        Then
            commission = 0.10 * 1000.0
            commission = commission + 0.15 * (sales - 1000.0)
        Else commission = 0.10 * sales
    EndIf
EndIf
Output("Commission is $", commission)
```



33

DD-Path Compression & Basis Path Testing Commission Problem



34

Reverse String Testing: In Class Exercise

```

1. String reverseString(String string) {
2.     String reverse = "";
3.     if(string != null) {
4.         for(int i = string.length()-1; i >= 0; i--) {
5.             reverse += string.charAt(i);
6.         }
7.     }
8.     return reverse;
9. }

```

35

Join Sorted Array Testing: In Class Exercise

```

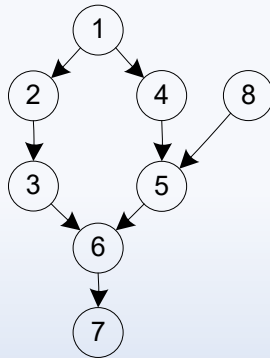
1. Vector<Integer> joinSortedArray(int[] a1, int[] a2) {
2.     Vector<Integer> s = new Vector<Integer>();
3.     int i1 = 0, i2 = 0;
4.     while(i1 < a1.length && i2 < a2.length) {
5.         if(a1[i1] < a2[i2])
6.             s.add(a1[i1++]);
7.         else
8.             s.add(a2[i2++]);
9.     }
10.    for(; i1 < a1.length; i1++)
11.        s.add(a1[i1]);
12.    for(; i2 < a2.length; i2++)
13.        s.add(a2[i2]);
14.    return s;
15.}

```

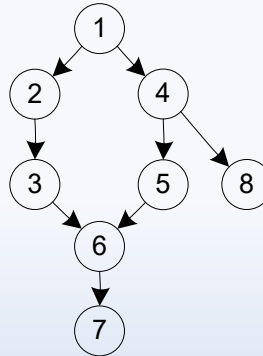
36

Structured Programming Concepts & Violations

Branching into a decision



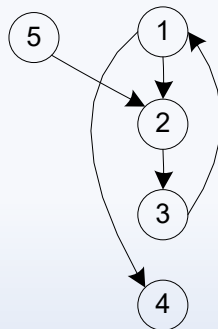
Branching out of a decision



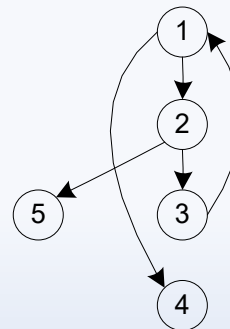
37

Structured Programming Concepts & Violations

Branching into a loop



Branching out of a loop



38

Class Overview

In Class:

- Code Coverage Testing
- Unit Testing Tools
- MC/DC Testing
- Decision Table-Based Testing

After Class:

- Office Hours: Code Coverage Inputs

Homework:

- 2/4: Symbolic Execution
- 2/18: Code Coverage Inputs
- 2/18: Midterm Selection (In-Class)
- 2/25: Midterm Assessment! (No Extensions or Late Submissions)

39

Midterm Assessment Choices

You may *individually* select from:

- A “traditional” exam, or
- Some other option? These *may* be possible:
 - A presentation* on an allowed topic?
 - A game of jeopardy?
 - Something else?

*No pre-recorded videos for the midterm assessment.

40

Code Coverage Testing

41

Coverage Based Testing

Design enough test cases such that:

- **Line coverage:** every line in a program is executed at least once.
- **Branch coverage:** each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage:** each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage:** each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.

42

Coverage Based Testing

Design enough test cases such that:

- **Multiple condition coverage:** all possible combinations of condition outcomes in each decision are invoked at least once.
- **MC/DC:** MCC *and* each condition is shown to independently affect the outcome of the decision.
- **Path coverage:** all execution paths are executed at least once:
 - A path is a unique sequence of branches from entry to exit.
 - Loops introduce an unbounded number of paths making path coverage criterion not practical.
 - Some paths are impossible to exercise due to relationships of data.
 - The number of executions of a loop may depend on the input variables and hence may not be possible to determine.

43

In Class Exercise

- **Line coverage:** every line in a program is executed at least once.
- **Branch coverage:** each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage:** each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage:** each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.
- **Multiple condition coverage:** all possible combinations of condition outcomes in each decision are invoked at least once.

```
int function(int a, int b, int c) {
    int branchCount = 0;
    if((a != 10) || (b == 10)) {
        branchCount++;
    }

    if((c == 10) && (b >= 10)) {
        branchCount++;
    }

    return branchCount;
}
```

44

• Line coverage: every line in a program is executed at least once. • Branch coverage: each decision/branch has a <i>true</i> and <i>false</i> outcome at least once. • Condition coverage: each condition in a decision/branch takes all possible outcomes at least once. • Branch/Condition coverage: each condition in a decision takes on all possible outcomes at least once, <i>and</i> each decision takes on all possible outcomes at least once. • Multiple condition coverage: all possible combinations of condition outcomes in each decision are invoked at least once.		<pre>int function(int a, int b, int c) { int branchCount = 0; if((a != 10) (b == 10)) { branchCount++; } if((c == 10) && (b >= 10)) { branchCount++; } return branchCount; }</pre>			
Test Case Inputs	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 10, b = 11, c = 11 a = 10, b = 10, c = 10					

45

• Line coverage: every line in a program is executed at least once. • Branch coverage: each decision/branch has a <i>true</i> and <i>false</i> outcome at least once. • Condition coverage: each condition in a decision/branch takes all possible outcomes at least once. • Branch/Condition coverage: each condition in a decision takes on all possible outcomes at least once, <i>and</i> each decision takes on all possible outcomes at least once. • Multiple condition coverage: all possible combinations of condition outcomes in each decision are invoked at least once.		<pre>int function(int a, int b, int c) { int branchCount = 0; if((a != 10) (b == 10)) { branchCount++; } if((c == 10) && (b >= 10)) { branchCount++; } return branchCount; }</pre>			
Test Case Inputs	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 11, b = 11, c = 10 a = 10, b = 10, c = 10					

46

• Line coverage: every line in a program is executed at least once. • Branch coverage: each decision/branch has a <i>true</i> and <i>false</i> outcome at least once. • Condition coverage: each condition in a decision/branch takes all possible outcomes at least once. • Branch/Condition coverage: each condition in a decision takes on all possible outcomes at least once, <i>and</i> each decision takes on all possible outcomes at least once. • Multiple condition coverage: all possible combinations of condition outcomes in each decision are invoked at least once.					
<pre> int function(int a, int b, int c) { int branchCount = 0; if((a != 10) (b == 10)) { branchCount++; } if((c == 10) && (b >= 10)) { branchCount++; } return branchCount; } </pre>					
Test Case Inputs	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 9, b = 10, c = 10 a = 10, b = 9, c = 11					

47

• Line coverage: every line in a program is executed at least once. • Branch coverage: each decision/branch has a <i>true</i> and <i>false</i> outcome at least once. • Condition coverage: each condition in a decision/branch takes all possible outcomes at least once. • Branch/Condition coverage: each condition in a decision takes on all possible outcomes at least once, <i>and</i> each decision takes on all possible outcomes at least once. • Multiple condition coverage: all possible combinations of condition outcomes in each decision are invoked at least once.					
<pre> int function(int a, int b, int c) { int branchCount = 0; if((a != 10) (b == 10)) { branchCount++; } if((c == 10) && (b >= 10)) { branchCount++; } return branchCount; } </pre>					
Test Cases	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 10, b = 10, c = 11 a = 9, b = 9, c = 10					

48

- **Line coverage:** every line in a program is executed at least once.
- **Branch coverage:** each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage:** each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage:** each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.
- **Multiple condition coverage:** all possible combinations of condition outcomes in each decision are invoked at least once.

```
int function(int a, int b, int c) {
    int branchCount = 0;
    if((a != 10) || (b == 10)) {
        branchCount++;
    }

    if((c == 10) && (b >= 10)) {
        branchCount++;
    }

    return branchCount;
}
```

Test Case Inputs	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 9, b = 10, c=10 a = 9, b = 9, c=10 a = 10, b = 10, c=9 a = 10, b = 9, c=9					

49

Coverage Based Testing

Test Cases	Line	Branch	Conditional	Branch/ Conditional	Multiple Conditional
a = 10, b = 11, c = 11 a = 10, b = 10, c = 10	Yes	Yes	No	No	No
a = 11, b = 11, c = 10 a = 10, b = 10, c = 10	Yes	No	No	No	No
a = 9, b = 10, c = 10 a = 10, b = 9, c = 11	Yes	Yes	Yes	Yes	No
a = 10, b = 10, c = 11 a = 9, b = 9, c = 10	No	No	Yes	No	No
a = 9, b = 10, c=10 a = 9, b = 9, c=10 a = 10, b = 10, c=9 a = 10, b = 9, c=9	Yes	Yes	Yes	Yes	Yes

50

Code Coverage vs Graph

- **Line coverage:** every line in a program is executed at least once.
- **Branch coverage:** each decision/branch has a *true* and *false* outcome at least once.
- **Condition coverage:** each condition in a decision/branch takes all possible outcomes at least once.
- **Branch/Condition coverage:** each condition in a decision takes on all possible outcomes at least once, *and* each decision takes on all possible outcomes at least once.
- **Multiple condition coverage:** all possible combinations of condition outcomes in each decision are invoked at least once.
- **G_{node}:** Every node in the graph is executed
- **G_{edge}:** Every edge in the graph is traversed during execution
- **G_{chain}:** Every chain in the graph is executed